

FAST MATRIX MULTIPLICATION ALGORITHMS

A Project Report Submitted
in Partial Fulfilment of the Requirements
for the Degree of

BACHELOR OF TECHNOLOGY
in
Mathematics and Computing

by

Boyapati V S S Pruthvi
(Roll No. 190123019)



to

DEPARTMENT OF MATHEMATICS
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI
GUWAHATI - 781039, INDIA

April 2023

CERTIFICATE

This is to certify that the work contained in this project report entitled “Fast Matrix Multiplication Algorithms” submitted by Boyapati V S S Pruthvi (Roll No.: 190123019) to the Department of Mathematics, Indian Institute of Technology Guwahati towards partial requirement of Bachelor of Technology in Mathematics and Computing has been carried out by him under my supervision.

It is also certified that this report is a survey work based on the references in the bibliography.

Turnitin Similarity: 16%

Guwahati - 781 039

April 2023

(Dr. Sagarmoy Dutta)

Project Supervisor

ABSTRACT

Matrix multiplication is one of the fundamental operations involved in mathematical computations. Hence, optimizing and increasing the efficiency of computing matrix multiplication is necessary and is of great interest. Until 1969, the naive schoolbook algorithm is thought to be optimal and can not be improved, but Strassen's discovery opened a new field of research. It is still an open problem in mathematics and computer science to find the fastest algorithm for matrix multiplication. Research developments through the past five decades have brought many novel ideas and methods to efficiently compute matrix multiplication. In this project, we discuss the development of those algorithms and present the working and ideas behind them. New approaches and ideas may be required for better understanding and to further decrease the exponent of matrix multiplication.

Key words: Matrix multiplication, Strassen's algorithm, rank, border rank, tensor, bilinear maps, trilinear maps, trilinear forms, Approximation algorithms, Bini's algorithm, Asymptotic sum inequality, tensor power, combinatorial argument, Coppersmith Winograd's algorithm

Contents

1	Introduction	1
1.1	Matrix multiplication	1
2	Strassen's Algorithm	4
2.1	2×2 matrices	4
2.2	$2^n \times 2^n$ matrices	5
2.3	Other square matrices	6
2.4	Complexity	6
3	Bilinear Maps	7
3.1	Model	8
3.2	Relationship between rank and the exponent	8
4	Trilinear Forms	11
4.1	Schonhage's theorem	12
4.2	Strassen's trilinear form	12
5	Introduction to CW Algorithm	15
5.1	Trilinear construction	15
5.2	Complexity	16

6	Approximation algorithms	17
6.1	Bini's algorithm	17
6.2	Symmetrization	19
7	Direct Sums of Matrix Multiplication Tensors	23
8	Combinatorial calculation of CW algorithm	26
	Bibliography	30

Chapter 1

Introduction

Matrix multiplication, as a fundamental mathematical operation requires to be computed very often. Its optimization requires efficient procedures in various stages of computation, from algorithm and software to implementation on the hardware. Many factors like algorithmic arithmetic complexity, communication complexity in case of distributed computing, vectorization and data transfer need to be studied and optimized. In this project, we are only interested in studying and improving arithmetic complexity. We start with a discussion on the development of efficient matrix multiplication methods and algorithms discovered till date.

1.1 Matrix multiplication

Definition 1.1.1. (Matrix Multiplication) The matrix multiplication of two matrices A of order $m \times n$ and B of order $n \times p$ is the matrix C of order $m \times p$ given by

$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}, \quad (1.1)$$

where a_{uv} , b_{uv} , c_{uv} are the u^{th} row- v^{th} column elements of A , B and C respectively.

In order to multiply matrices A and B as stated in 1.1.1, if we choose to multiply the entries of A and B as defined, then we end up in an algorithm which is generally referred to as *naive* or *schoolbook* algorithm. Since n multiplications and $n - 1$ additions are required to compute a single element of C , we require mnp number of multiplications and $mp(n - 1)$ number of additions in total, to compute C .

If A , B are square matrices of order $n \times n$, then the total number of multiplications required is n^3 and the total number of additions required is $n^3 - n^2$.

Remark 1.1.2. The total number arithmetic operations required to compute the product of two square matrices of order $n \times n$ is $O(n^3)$.

From now on, we discuss the algorithms related to square matrices unless specified otherwise. Looking at the naive algorithm, one may wonder if it can be optimized. It might seem that the algorithm is optimal. But in fact it is not. In the year 1969, Volker Strassen showed that the product of two square matrices of order n can be computed in less than $O(n^3)$ operations [5]. It is the breakthrough to this field of finding optimized matrix multiplication algorithms.

Strassen showed that the multiplication of a 2×2 matrix can be computed using 7 multiplications instead of the usual 8. Moreover Strassen's algorithm can be applied to block matrices and can be applied on those constituent submatrices treating them similar to real numbers. Hence the product of any two matrices of order $2^n \times 2^n$ can be computed using 7 multiplications of $2^{n-1} \times 2^{n-1}$ matrices. Hence Strassen's algorithm runs in $O(7^{\log_2 n})$ that is, $O(n^{\log_7 2}) \approx O(n^{2.807})$. A detailed explanation on how Strassen's algorithm is

constructed and works is presented in Chapter 2.

The discovery of Strassen's algorithm led to the discovery of better algorithms. Better algorithms in terms of asymptotic arithmetic complexity were designed later in 1978, 1979, 1980, 1981 etc.

As of October 2020, the best asymptotic arithmetic complexity achieved is $O(n^{2.37286})$ [1].

The algorithms are based on various fields of bilinear maps, trilinear forms of tensors among others.

The next chapter describes about Strassen's algorithm and subsequent chapters discuss bilinear maps, trilinear forms etc.

Chapter 2

Strassen's Algorithm

2.1 2×2 matrices

Let two 2×2 matrices A, B be as follows. Let C be the product of matrices A and B , $C = AB$.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}, C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

Define $m_1, m_2 \dots m_7$ as follows:

$$m_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$m_2 = (a_{21} + a_{22})b_{11}$$

$$m_3 = a_{11}(b_{12} - b_{22})$$

$$m_4 = a_{22}(b_{21} - b_{11})$$

$$m_5 = (a_{11} + a_{12})b_{22}$$

$$m_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$m_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

Then the entries of C are as follows:

$$c_{11} = m_1 + m_4 - m_5 + m_7$$

$$c_{12} = m_3 + m_5$$

$$c_{21} = m_2 + m_4$$

$$c_{22} = m_1 - m_2 + m_3 + m_6$$

It can be seen that we require only 7 multiplications instead of the usual 8 to perform the multiplication. Now we extend this algorithm to $2^n \times 2^n$ matrices [5].

2.2 $2^n \times 2^n$ matrices

If the matrices are of order $2^n \times 2^n$, then we partition each matrix into 4 submatrices of order $2^{n-1} \times 2^{n-1}$. Strassen's algorithm works on blocks of matrices too. Hence the blocks of matrices can be treated as if they are real numbers and the multiplication can be carried out.

This requires 7 multiplications of $2^{n-1} \times 2^{n-1}$ matrices. Similarly, this process is continued until a 2×2 matrix is reached.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

2.3 Other square matrices

Let the order of the given square matrices be n . If the product of two square matrices whose order is not a power of 2 is required then the given matrices can be adjusted to the least power of 2 that is greater than n by adding a padding of zeroes to the right and the bottom. After the multiplication of the modified matrices is computed using Strassen's algorithm, the required matrix can be retrieved by removing the padding zeroes.

2.4 Complexity

For a matrix of order n , since the matrix is reduced to an order half the initial order, the total number of such reductions would be of the order $\log_2 n$. But each such reduction includes 7 multiplications to be performed. Hence the complexity would be the order of $7^{\log_2 n}$. Hence,

$$Complexity = O(7^{\log_2 n}) = O(n^{\log_2 7}) \approx O(n^{2.807}) \quad (2.1)$$

It is later proved that 7 is the optimal number of multiplications required to multiply two 2×2 matrices.

Chapter 3

Bilinear Maps

Definition 3.0.1. (Bilinear map) Let U, V and X be vector spaces on $\mathbb{F}$.

Then $\phi : U \times V \rightarrow X$ is called a bilinear map if

$$\phi(\lambda_1 u_1 + \lambda_2 u_2, \mu_1 v_1 + \mu_2 v_2) = \lambda_1 \mu_1 \phi(u_1 v_1) + \lambda_1 \mu_2 \phi(u_1 v_2) + \lambda_2 \mu_1 \phi(u_2 v_1) + \lambda_2 \mu_2 \phi(u_2 v_2) \quad (3.1)$$

$\forall u_1, u_2 \in U, v_1, v_2 \in V$ and $\lambda_1, \lambda_2, \mu_1, \mu_2 \in \mathbb{F}$.

Matrix multiplication of two matrices is a bilinear map on the vector space of matrices. For $n \times n$ matrix multiplication, the vector space is $\mathbb{F}^{n \times n}$. From now on, we denote the bilinear map of matrix multiplication of $n \times n$ matrices as $\phi = \langle n, n, n \rangle$.

Definition 3.0.2. (Rank) The rank of a bilinear map ϕ is the smallest $r \in \mathbb{N}$ such that

$$\phi(u, v) = \sum_i^r f_i(u) g_i(v) x_i, \quad (3.2)$$

where $x_i \in X$. Here f, g are linear maps from U, V (respectively) to X . We denote rank of ϕ as $R(\phi)$.

Definition 3.0.3. (The exponent of matrix multiplication, ω) The exponent

of matrix multiplication is defined as,

$$\omega = \inf\{p \in \mathbb{R} : M(n) = O(n^p)\}, \quad (3.3)$$

where $M(n)$ is the number of multiplications required to perform the multiplication of two square matrices of order n .

Hence, ω is vaguely the minimum exponent of p such that $M(n)$ is $O(n^p)$.

3.1 Model

We assume that, in order to compute the product of two matrices, we perform a number of additions/subtractions, then take their products and finally add/subtract the products to find the entries of the product matrix, like the Strassen's algorithm. Also, we assume that the asymptotic complexity of number of additions is not more than that of multiplications.

3.2 Relationship between rank and the exponent

Definition 3.2.1. (Exponent of rank) The exponent of the rank of the bilinear map $\langle n, n, n \rangle$ denoted by ω' is defined as

$$\omega' = \inf\{p \in \mathbb{R} : R(\langle n, n, n \rangle) = O(n^p)\} \quad (3.4)$$

Let $\phi = \langle n, n, n \rangle$. Then

$$\phi(u, v) = \sum_{i=1}^{|M(n)|} f_i(u, v) g_i(u, v) x_i \quad (3.5)$$

for some bilinear maps f, g . The reason for why f and g must be bilinear goes as follows: since ϕ is bilinear, and we are performing arithmetic operations of addition and multiplications only, we must have all the intermediate steps of the computation to be bilinear. Hence f and g must also be bilinear.

Since f and g are bilinear with respect to their entries, we have

$$\phi(u, v) = \sum_{i=1}^{|M(n)|} (f_i(u, 0) + f_i(0, v))(g_i(u, 0) + g_i(0, v))x_i \quad (3.6)$$

$$= \sum_{i=1}^{|M(n)|} f_i(u, 0)g_i(0, v)x_i + \sum_{i=1}^{|M(n)|} f_i(0, v)g_i(u, 0)x_i \quad (3.7)$$

The terms,

$$\sum_{i=1}^{|M(n)|} f_i(u, 0)g_i(u, 0)x_i, \quad \sum_{i=1}^{|M(n)|} f_i(0, v)g_i(0, v)x_i \quad (3.8)$$

are not bilinear, hence if are non-zero will make the map ϕ non-bilinear. Hence the terms in 3.8 must be zero. From the above equation, we can say that $R(\langle n, n, n \rangle) \leq 2M(n)$. Hence $\omega' \leq \omega$. Now, similar to Strassen's algorithm, we use the block decomposition method to observe that

$$M(n^m) \leq rM(n^{m-1}) + O((n^{m-1})^2) \quad (3.9)$$

This is because, the multiplication of two $n^m \times n^m$ matrices can be seen as a multiplication of a $n \times n$ matrix whose entries are $n^{m-1} \times n^{m-1}$ matrices. The recurrence relation yields

$$M(n^m) = O(r^m) \quad (3.10)$$

Hence, for a given k ,

$$M(k) = O(n^{\log_n k}) = O(r^{\log_n k}) = O(k^{\log_n r}) = O(k^{\omega'}) \quad (3.11)$$

Hence $\omega \leq \omega'$. Since, it is also concluded that $\omega' \leq \omega$, we have $\omega = \omega'$.

Remark 3.2.2. The exponent of matrix multiplication equals the exponent of the rank of the bilinear map that represents matrix multiplication i.e., $\omega = \omega'$. ([4], Prop. 15.1)

In the forthcoming chapters, this remark is used to find an upper bound on the exponent of matrix multiplication by using the bi/trilinear maps that represent matrix multiplication.

Remark 3.2.3. From the definition of ω' , and 3.2.2, we have $R(\langle n, n, n \rangle) \leq r \Rightarrow n^{\omega'} \leq r \Leftrightarrow n^{\omega} \leq r$. ([4], Prop. 15.3)

Chapter 4

Trilinear Forms

The bilinear map $\phi : U \times V \rightarrow X$ can be expressed as $\phi = \phi' \circ t$, where $t : U \times V \rightarrow U \otimes V$ is the tensor product of U, V and $\phi' : U \otimes V \rightarrow X$ is a linear map. This is generally referred to as the universality of the tensor product.

Likewise, given a bilinear map $\phi : U \times V \rightarrow X$, let a trilinear map $\bar{\phi} : U \times V \times X \rightarrow W$ be defined where $\bar{\phi} = \bar{\phi}' \circ \bar{t}$. $\bar{t} : U \times V \times X \rightarrow U \otimes V \otimes X$ and $\bar{\phi}' : U \otimes V \otimes X \rightarrow W$ are the corresponding tensor and linear map respectively. Hence for $\langle n, n, n \rangle$ also, we can define a trilinear map with the corresponding tensor $\bar{t} : U^3 \rightarrow U^{\otimes 3}$ where U is the vector space of $n \times n$ matrices. The tensor can be understood like more of an identity among the two operand matrices and the product matrix. Hence the term trilinear form is used instead of trilinear map hereon.

The rank of a tensor t is similar to that of a bilinear map, that is, it is the minimum number of tensors required to be summed to express t . We now proceed to define border rank of a tensor and state some results related to border rank.

Definition 4.0.1. (Border rank) The border rank of a tensor t is the smallest

number $r \in \mathbb{N}$ such that a sequence of tensors in the space of t of rank at most r converges to t . We denote the border rank of t as $\underline{R}(t)$.

Remark 4.0.2. $\underline{R}(t) \leq R(t)$.

Remark 4.0.3. $\underline{R}(\langle n, n, n \rangle) \leq r \Rightarrow n^\omega \leq r$. ([4], Prop. 15.10)

In the year 1979, Bini et al. proved that $\underline{R}(\langle 12, 12, 12 \rangle) \leq 1000$. Hence $\omega \leq \log_{12} 1000 \approx 2.780$. [3]

Here below, we state an important theorem without proof.

4.1 Schonhage's theorem

Theorem 4.1.1. (*Schonhage's theorem*) $\underline{R}(\bigoplus_{i=1}^s \langle n, n, n \rangle) \leq r \Rightarrow sn^\omega \leq r$. ([4], Prop. 15.11)

4.0.3 and this theorem say that, if we succeed at producing a construction that allows for an efficient approximation to matrix multiplication, then we can also compute it exactly efficiently. Moreover, the theorem also states how much of such efficiency in exact computation of matrix multiplication is obtainable through a certain way of approximating it. If s independent sets of matrices are approximately computed for matrix multiplication, then the upper bound of the exponent of matrix multiplication can be decreased by $\log(s)$.

4.2 Strassen's trilinear form

Now, we present the trilinear form of Strassen's algorithm.

$$\begin{aligned} & (A_{11} + A_{22})(B_{11} + B_{22})(C_{11} + C_{22}) \\ & + (A_{21} + A_{22})B_{11}(C_{21} - C_{22}) \end{aligned}$$

$$\begin{aligned}
& + A_{11} (B_{12} - B_{22}) (C_{12} + C_{22}) \\
& + A_{22} (B_{21} - B_{11}) (C_{11} + C_{21}) \\
& + (A_{11} + A_{12}) B_{22} (-C_{11} + C_{12}) \\
& + (A_{21} - A_{11}) (B_{11} + B_{12}) C_{22} \\
& + (A_{12} - A_{22}) (B_{21} + B_{22}) C_{11} \\
& \equiv (A_{11} B_{11} + A_{12} B_{21}) C_{11} \\
& + (A_{11} B_{12} + A_{12} B_{22}) C_{12} \\
& + (A_{21} B_{11} + A_{22} B_{21}) C_{21} \\
& + (A_{21} B_{12} + A_{22} B_{22}) C_{22}
\end{aligned}$$

This trilinear identity in fact describes the whole construction of Strassen's algorithm. Since it is an identity, the coefficients of C_{ij} on both the sides must be equal. This gives the complete algorithm to compute C . For example, consider C_{11} .

$$\begin{aligned}
\text{Coefficient of } C_{11} \text{ on LHS} &= (A_{11} + A_{22})(B_{11} + B_{22}) + A_{22}(B_{21} - B_{11}) - \\
& (A_{11} + A_{12})B_{22} + (A_{12} - A_{22})(B_{21} + B_{22}) = M_1 + M_4 - M_5 + M_7 \text{ (refer 2.2)}.
\end{aligned}$$

$$\text{Coefficient of } C_{11} \text{ on RHS} = (A_{11} B_{11} + A_{12} B_{21}) = C_{11}.$$

Hence, equating the coefficients on both the sides gives $C_{11} = M_1 + M_4 - M_5 + M_7$, which is what exactly stated by Strassen's algorithm. The rank of the above trilinear form is 7 since 7 is the optimal number of multiplications required and hence atleast 7 terms must be used on the LHS to express the identity. Hence from 3.2.3, we can deduce that $\omega \leq \log_2 7$ since $R(\langle 2, 2, 2 \rangle) \leq 7$.

It is now evident that the rank/border rank of a trilinear identity/form can be used to find an upper bound on ω . Since Strassen's algorithm is

exact, we deal with trilinear identity and its rank. But as we have already seen, even approximating algorithms do the job. Hence, if an approximate trilinear form can be constructed and if its border rank can be calculated, then we can find an upper bound on ω . This leads us to the Coppersmith Winograd's (referred to as CW hereon) trilinear construction.

Chapter 5

Introduction to CW Algorithm

5.1 Trilinear construction

The following trilinear approximation is constructed:

$$\begin{aligned} & \epsilon \sum_{i=1}^q \left(x_0^{[0]} + \epsilon x_i^{[1]} \right) \left(y_0^{[0]} + \epsilon y_i^{[1]} \right) \left(z_0^{[0]} + \epsilon z_i^{[1]} \right) \\ & \left(x_0^{[0]} + \epsilon^2 \sum_{i=1}^q x_i^{[1]} \right) \left(y_0^{[0]} + \epsilon^2 \sum_{i=1}^q y_i^{[1]} \right) \left(z_0^{[0]} + \epsilon^2 \sum_{i=1}^q z_i^{[1]} \right) \\ & + (1 - q\epsilon) x_0^{[0]} y_0^{[0]} z_0^{[0]} \\ & = \epsilon^3 \sum_{i=1}^q \left(x_0^{[0]} y_i^{[1]} z_i^{[1]} + x_i^{[1]} y_0^{[0]} z_i^{[1]} + x_i^{[1]} y_i^{[1]} z_0^{[0]} \right) + O(\epsilon^4) = t + O(\epsilon^4). ([2]) \end{aligned}$$

This symmetric approximation gives a guide on developing more rigorous construction that is more relevant to $n \times n$ matrix multiplication and that contains as few number of terms as possible. If we observe the RHS of the identity, we can see that there is only one possible $x_0^{[0]}$ but q different $y_i^{[1]}$ (and $z_i^{[1]}$ s). Similarly, there is only one possible $y_0^{[0]}$, $z_0^{[0]}$ with q different $x_i^{[1]}$ s. That is this construction has the sums on the RHS representing $\langle 1, 1, q \rangle, \langle q, 1, 1 \rangle, \langle 1, q, 1 \rangle$. That is this corresponds to scalar multiplication of a row vector ($\langle 1, 1, q \rangle$ represents the multiplication of 1×1 matrix and $1 \times q$ matrix). In order to scale up the construction to matrices of order $n \times n$, the tensor representing the identity will be taken to the tensor power

of $3N$ for some $N \in \mathbb{N}$. And from there rigorous restrictions are imposed on how to modify and choose among the terms to obtain minimum number of terms possible. This is done through the concepts of hashing based on dense Salem-Spencer sets (dense sets of numbers in which no three numbers form an arithmetic progression, also called as 3-AP-free sets) among others [2]. The details of how the number of terms are restricted and the counting argument will be dealt with later.

5.2 Complexity

The problem of finding the number of terms in the final modified identity falls into the realm of combinatorics. Based on the restrictions imposed and with the appropriate values of r, s and n , Schonhage's theorem (4.1), implies that [2]

$$\left(\frac{27}{4}\right)q^\omega \leq (q+2)^3. \quad (5.1)$$

Optimizing the inequality to get the least upper bound on ω gives (at $q = 8$) [2],

$$\omega \leq \log_8 \frac{4000}{27} = 2.404. \quad (5.2)$$

There is an another complicated construction by Coppersmith and Winograd which produces $\omega \leq 2.376$. [2]

In the following chapters, we explain how the bound is calculated. To understand the calculation and the ideas behind better, we cover topics on simpler algorithms that provide a bound greater than 2.404. We use all such ideas together in CW algorithm to obtain the bound 2.404.

Chapter 6

Approximation algorithms

6.1 Bini's algorithm

In the year 1979, Bini et al. [3] showed that a partial 2×2 matrix can be multiplied with a regular 2×2 matrix, not exactly, but approximately. The algorithm proposed can be put into a trilinear form. The matrix product is

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & 0 \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

The algorithm in its trilinear form is as follows, and contains 11 variables.

$$\begin{aligned} & \epsilon (a_{11}b_{11}c_{11} + a_{11}b_{12}c_{12} + a_{12}b_{21}c_{11} + a_{12}b_{22}c_{12} + a_{21}b_{11}c_{21} + a_{21}b_{12}c_{22}) + \\ & \epsilon^2 (a_{11}b_{22}c_{12} + a_{11}b_{11}c_{21} + a_{12}b_{21}c_{22} + a_{21}b_{21}c_{22}) = \\ & (a_{12} + \epsilon a_{11}) (b_{12} + \epsilon b_{22}) c_{12} + \\ & (a_{21} + \epsilon a_{11}) b_{11} (c_{11} + \epsilon c_{21}) - \\ & a_{12}b_{12} (c_{11} + c_{12} + \epsilon c_{22}) - \\ & a_{21} (b_{11} + b_{12} + \epsilon b_{21}) c_{11} + \\ & (a_{12} + a_{21}) (b_{12} + \epsilon b_{21}) (c_{11} + \epsilon c_{22}) \end{aligned}$$

The way to abstract the algorithm from the identity is similar to that described in Chapter 4, for Strassen's algorithm. Comparing the coefficients of c_{ij} in the identity gives the algorithm to compute the entry c_{ij} approximately. For example, comparing coefficients of c_{22} on both the sides gives

$$\epsilon(a_{21}b_{12}) + \epsilon^2(a_{21}b_{21}) = \epsilon a_{12}b_{12} + \epsilon(a_{12} + a_{21})(b_{12} + \epsilon b_{21}) \quad (6.1)$$

The entry c_{22} is itself equal to $a_{21}b_{12}$ numerically. Hence the algorithmic step to compute c_{22} is

$$c_{22} = \frac{a_{12}b_{12} + (a_{12} + a_{21})(b_{12} + \epsilon b_{21})}{\epsilon} - \epsilon(a_{21}b_{21}) \quad (6.2)$$

This computation gives an approximate value for c_{22} when ϵ is chosen to be arbitrarily small and positive. This identity can be used to compute approximately the matrix multiplication of a regular 3×2 matrix with a regular 2×2 matrix.

The multiplication of a 3×2 and a 2×2 matrix can be represented as

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \left(\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \right) \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

$$= \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \\ c_{31} & c_{32} \end{bmatrix}$$

Taking two copies of Bini's identity, one with $a_{11}, a_{12}, a_{21}, b_{11}, b_{12}, b_{21}, b_{22}, c_{11}, c_{12}, c_{21}, c_{22}$ and another with $a_{22}, a_{31}, a_{32}, b_{11}, b_{12}, b_{21}, b_{22}, c_{21}, c_{22}, c_{31}, c_{32}$ (each

with 11 variables and 5 terms on the right-hand side(RHS)) and adding them with a little adjustment made to the indices to account for the orientation of the two matrices involving a variables, we will get an approximation algorithm for the $(3, 2, 2)$ problem with 10 terms on the RHS. Let t be the tensor $\langle 3, 2, 2 \rangle$ from this identity. Then the identity can be expressed as

$$\epsilon t + \epsilon^2 t'(\epsilon) = \sum_{i=1}^{10} u_i(\epsilon) \otimes v_i(\epsilon) \otimes w_i(\epsilon) \quad (6.3)$$

This implies that

$$R(\langle 3, 2, 2 \rangle + \epsilon T') \leq 10 \text{ for any } \epsilon > 0 \quad (6.4)$$

This means that the bound on the rank of a tensor that is arbitrarily close to $\langle 3, 2, 2 \rangle$ is obtained, but not on $\langle 3, 2, 2 \rangle$ itself. This concept of the rank of an arbitrarily close tensor can be established as what is called as border rank, which has already been defined in Chapter 4. Hence, $\underline{R}(\langle 3, 2, 2 \rangle) \leq 10$.

6.2 Symmetrization

Since for two matrices A and B , $(AB)^t = B^t A^t$, it is evident that the $(3, 2, 2)$ problem is analogous to the $(2, 2, 3)$ problem and therefore, $\underline{R}(\langle 3, 2, 2 \rangle) = \underline{R}(\langle 2, 2, 3 \rangle)$. Similarly, the $(2, 3, 2)$ problem would be analogous with a slight modification that needs to be done in order to incorporate the positions of the entries. Hence, without a rigorous proof, we state that

$$\underline{R}(\langle 3, 2, 2 \rangle) = \underline{R}(\langle 2, 3, 2 \rangle) = \underline{R}(\langle 2, 2, 3 \rangle). \quad (6.5)$$

This kind of equality holds for any $\underline{R}(< n, m, p >)$ with all the possible permutations. Hence taking a tensor product of 3 identities of Bini, involving $< 3, 2, 2 >$, $< 2, 3, 2 >$, $< 2, 2, 3 >$, we get,

$$\underline{R}(< 12, 12, 12 >) \leq 1000. \quad (6.6)$$

This can be represented as follows [3] :

$$\left[\begin{array}{c|c} 4 \times 6 & 4 \times 6 \\ \hline 4 \times 6 & 4 \times 6 \\ \hline 4 \times 6 & 4 \times 6 \end{array} \right]_{12 \times 12} \times \left[\begin{array}{c|c} 6 \times 6 & 6 \times 6 \\ \hline 6 \times 6 & 6 \times 6 \end{array} \right]_{12 \times 12} = \left[\begin{array}{c|c} 4 \times 6 & 4 \times 6 \\ \hline 4 \times 6 & 4 \times 6 \\ \hline 4 \times 6 & 4 \times 6 \end{array} \right]_{12 \times 12}$$

In the multiplication of two 12×12 matrices, we partition the matrices into blocks so that, the problem becomes $(3, 2, 2)$ with resulting multiplications to be performed on block matrices of size 4×6 and 6×6 . Since the algorithm for $(3, 2, 2)$ is known, the newly formed problem would be $(4, 6, 6)$. This in turn can be viewed as $(2, 3, 2)$ with resulting multiplications to be performed between 2×2 and 2×3 sized block matrices as follows:

$$\left[\begin{array}{c|c|c} 2 \times 2 & 2 \times 2 & 2 \times 2 \\ \hline 2 \times 2 & 2 \times 2 & 2 \times 2 \end{array} \right]_{4 \times 6} \times \left[\begin{array}{c|c} 2 \times 3 & 2 \times 3 \\ \hline 2 \times 3 & 2 \times 3 \\ \hline 2 \times 3 & 2 \times 3 \end{array} \right]_{6 \times 6} = \left[\begin{array}{c|c} 2 \times 3 & 2 \times 3 \\ \hline 2 \times 3 & 2 \times 3 \end{array} \right]_{4 \times 6}$$

Since the algorithm for the $(2, 3, 2)$ problem is known, the problem has been condensed to the $(2, 2, 3)$ problem, whose algorithm is also known.

$$\left[\begin{array}{c|c} \cdot & \cdot \\ \hline \cdot & \cdot \end{array} \right]_{2 \times 2} \times \left[\begin{array}{c|c|c} \cdot & \cdot & \cdot \\ \hline \cdot & \cdot & \cdot \end{array} \right]_{2 \times 3} = \left[\begin{array}{c|c|c} \cdot & \cdot & \cdot \\ \hline \cdot & \cdot & \cdot \end{array} \right]_{2 \times 3}$$

Hence we have constructed the algorithm for the $(12, 12, 12)$ problem completely. This process of using the tensor product to construct the algorithm for the multiplication of square matrices of higher dimension from an algorithm for the multiplication of rectangular matrices of lower dimensions is called **symmetrization**.

This is true for any general $\langle n, m, p \rangle$. If we have $\underline{R}(\langle n, m, p \rangle) \leq r$, then it is true that $\underline{R}(\langle nmp, nmp, nmp \rangle) \leq r^3$.

Remark 6.2.1. $\underline{R}(\langle n, m, p \rangle) \leq r \implies \underline{R}(\langle nmp, nmp, nmp \rangle) \leq r^3$.

Remark 6.2.2. $\omega \leq 3 \log_{nmp} \underline{R}(\langle n, m, p \rangle)$.

We will shortly see that all the above results stated in terms of $\underline{R}$ can be used for R also since exact algorithms can be constructed from approximation algorithms with no extra cost asymptotically.

From the above argument, we have

$$\underline{R}(\langle 12, 12, 12 \rangle) \leq 1000. \quad (6.7)$$

As we have stated in Chapter 4, in *Remark 4.0.3*, we have

$$\omega \leq \log_{12} 1000 \approx 2.78. \quad (6.8)$$

To see how this works out, take the tensor power of the identity that is obtained (which has $\epsilon^3(\langle 12, 12, 12 \rangle) + O(\epsilon^4)$ on the left-hand side (LHS)) to the power of some natural number N . Then we get an identity with $\epsilon^{3N}(\langle 12^N, 12^N, 12^N \rangle) + O(\epsilon^{3N+1})$ on the LHS and a sum of terms which are products of linear combinations of variables of types a, b and c . But each linear combination may have $k_1 \epsilon^{k_2}$ as a coefficient where $k_2 \in \{0, 1, \dots, 3N + 1\}$. Since we have to collect terms having ϵ^{3N} to find $\langle 12^N, 12^N, 12^N \rangle$, we will have at most $\binom{3N+2}{2}$ possible ways of picking such coefficients from the

terms containing linear combinations of a and b to obtain $\langle 12^N, 12^N, 12^N \rangle$. Hence even though the identity contains ϵ , if we want exact algorithm, then it does not take more than $O(N^2)$ operations per each step of approximate algorithm.

Hence we have,

$$\omega \leq \log_{12^N} O(N^2) 1000^N \rightarrow \log_{12^N} 1000^N \approx 2.78 \text{ as } N \rightarrow \infty. \quad (6.9)$$

This provides an intuitive explanation of why bounding $\underline{R}(\langle n, m, p \rangle)$ is sufficient to bound ω and obtain the required algorithm. The approximate nature of the algorithm does not make a difference in the limiting sense. We will discuss in due course that even if we are able to bound the border rank of a sum of independent matrix multiplication tensors, then too, we obtain a bound on ω .

Chapter 7

Direct Sums of Matrix Multiplication Tensors

By theorem 4.1.1, we have $\omega \leq \log_n \frac{R(\oplus_{i=1}^s \langle n, n, n \rangle)}{s}$. For any general n, m and p , we have

$$\omega \leq 3 \log_{nmp} \frac{R(\oplus_{i=1}^s \langle n, m, p \rangle)}{s} \quad (7.1)$$

Here, we state a similar theorem without proof in a general sense.

Theorem 7.0.1. (*Asymptotic sum inequality*) For any natural numbers n_i, m_i and p_i ,

$$\Sigma_i(n_i, m_i, p_i)^{\frac{\omega}{3}} \leq \underline{R}(\oplus_{i=1}^s \langle n_i, m_i, p_i \rangle). \quad (7.2)$$

Theorem 4.1.1 and 7.1 are special cases of 7.0.1. If $n_i = n, m_i = m, p_i = p \forall i$, then

$$s(nmp)^{\frac{\omega}{3}} \leq \underline{R}(\oplus_{i=1}^s \langle n, m, p \rangle) \implies \omega \leq 3 \log_{nmp} \frac{R(\oplus_{i=1}^s \langle n, m, p \rangle)}{s} \quad (7.3)$$

To see the working principle behind theorem 4.1.1, consider matrices of very large order $n^k \times n^k$, where n and k are very large natural numbers. Let

$\underline{R}(\oplus_{i=1}^s < n, n, n >) \leq r$. If we require an algorithm for (n^k, n^k, n^k) , that is to multiply two square matrices of size n^k , since we can multiply s number of independent (n, n, n) matrix multiplications simultaneously, with r operations, treat the $n^k \times n^k$ matrices as $n \times n$ matrices with $n^{k-1} \times n^{k-1}$ block matrices as the entries. If we make use of the tensor identity of $\oplus_{i=1}^s < n, n, n >$ with only one set of variables and setting other $s - 1$ sets of variables to zero, we can multiply the single instance of multiplication of the two $n \times n$ matrices, which results in r number of $(n^{k-1}, n^{k-1}, n^{k-1})$ matrix multiplications. Now the tensor identity $\oplus_{i=1}^s < n, n, n >$ can be fully used to calculate r number of $(n^{k-1}, n^{k-1}, n^{k-1})$ matrix multiplications and we only require $\lceil \frac{r}{s} \rceil$ multiplications because each s number of such matrix products can be grouped and multiplied simultaneously by using $\oplus_{i=1}^s < n, n, n >$. Hence asymptotically, our identity can be fully used giving us

$$\omega \leq \log_{n^k} \left(\frac{r}{s} \right)^k \quad (7.4)$$

which implies that

$$\omega \leq \log_n \frac{r}{s} \quad (\text{ or } sn^\omega \leq r). \quad (7.5)$$

Schonhage showed that,

$$\underline{R}(< 4, 1, 4 > \oplus < 1, 9, 1 >) \leq 17. \quad (7.6)$$

It is known that $\underline{R}(< 4, 1, 4 >) \leq 16$ and $\underline{R}(< 1, 9, 1 >) \leq 9$. But the inequality 7.6 shows that an outer product of vectors of length 4 and an inner product of vectors of length 9 can be computed approximately in just $4^2 + 1$ multiplications. The use of 7.0.1 on inequality 7.6 gives us that

$$16^{\frac{\omega}{3}} + 9^{\frac{\omega}{3}} \leq 17. \quad (7.7)$$

Let τ be the real number such that $16^\tau + 9^\tau = 17$. Then the bound on ω can be expressed as

$$\frac{\omega}{3} \leq \tau \implies \omega \leq 3\tau. \quad (7.8)$$

We have $\tau \approx 0.85$. Hence,

$$\omega \leq 2.55. \quad (7.9)$$

Theorem 7.0.1 serves as a gateway for many algorithms. It states that if one can bound the rank of direct sums of tensor products of arbitrary matrix multiplications, then we can have a bound on ω , that is there is no constraint on the sizes of the matrix multiplication tensors that are being summed up. Hence, one key idea that has emerged is to take a tensor power of a tensor identity representing different matrix products that need not be independent (that is have shared variables) and then somehow extract independent matrix product tensors out of the resulting tensor power expansion.

The process of obtaining sets of independent variables from dependent variables operates by zeroing out some of the variables present in the higher tensor power. For example, let there be a shared variable x , that in its higher tensor power manifests in variables $x_i, i \in \{1, 2, \dots, 8\}$ and let t_1, t_2, t_3 are the tensors that share the variable before zeroing out. While zeroing out, each tensor will only be provided a set of values of i from $\{1, 2, \dots, 8\}$ so that there will not be any shared variables further, say x_1 and x_2 are used by t_1 (rest zeroed out), x_3 by t_2 , x_6 and x_8 by t_3 . This technique of zeroing out is used in CW algorithm.

Chapter 8

Combinatorial calculation of CW algorithm

The CW's algorithm starts with the identity

$$\begin{aligned} & \epsilon \sum_{i=1}^q (x_0^0 + \epsilon x_i^1) (y_0^0 + \epsilon y_i^1) (z_0^0 + \epsilon z_i^1) \\ & (x_0^0 + \epsilon^2 \sum_{i=1}^q x_i^1) (y_0^0 + \epsilon^2 \sum_{i=1}^q y_i^1) (z_0^0 + \epsilon^2 \sum_{i=1}^q z_i^1) \\ & + (1 - q\epsilon) x_0^0 y_0^0 z_0^0 \\ & = \epsilon^3 \sum_{i=1}^q (x_0^0 y_i^1 z_i^1 + x_i^1 y_0^0 z_i^1 + x_i^1 y_i^1 z_0^0) + O(\epsilon^4) = t + O(\epsilon^4). ([2]) \end{aligned}$$

The above identity represents the approximation algorithm of the following matrix multiplications

$$[x_0^0] \times [y_1^1 y_2^1 \dots y_q^1] = [z_1^1 z_2^1 \dots z_q^1]$$

$$[x_1^1 x_2^1 \dots x_q^1]' \times [y_0^0] = [z_1^1 z_2^1 \dots z_q^1]'$$

$$[x_1^1 x_2^1 \dots x_q^1] \times [y_1^1 y_2^1 \dots y_q^1]' = [z_0^0]$$

Observe that all sets of variables are shared variables. To make sets of

independent variables, we take the tensor power of the above tensor identity to a natural number N . The LHS of the above identity can be represented as

$$\epsilon^3(< 1, 1, q >^{0,1,1} \oplus < q, 1, 1 >^{1,0,1} \oplus < 1, q, 1 >^{1,1,0}) + O(\epsilon^4). \quad (8.1)$$

The superscripts on the tensors indicate the superscripts of the variables that are being used in the tensors. A small definition would be handy for the following arguments.

Definition 8.0.1. (Volume of a tensor) The volume of a tensor $< n, m, p >$ is defined as $V = nmp$.

By taking the tensor in 8, to the power N , we have on the LHS,

$$\epsilon^{3N}(T) + O(\epsilon^{3N+1}), \quad (8.2)$$

where T is the sum of tensors each of volume q^N . Let each term of t be obtained from a number of tensors with superscript $0, 1, 1$, b number of tensors with superscript $1, 0, 1$, c number of tensors with superscript $1, 1, 0$, where $a + b + c = N$. From hereon in this argument, we refer to tensors with their superscripts.

For example, let $N = 5$. Then, the following tensor is a term in T .

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Here, $a = 1$, $b = 2$ and $c = 2$. The above term can be represented as 11101, 01110, 10011.

The identity shows that

$$\underline{R}(< 1, 1, q >^{0,1,1} \oplus < q, 1, 1 >^{1,0,1} \oplus < 1, q, 1 >^{1,1,0})^N \leq (q+2)^N. \quad (8.3)$$

Now, if suppose we are able to extract S_N number of tensors from T with independent variables, then from 7.0.1, we have

$$S_N q^{N\frac{\omega}{3}} \leq (q+2)^N \implies \omega \leq 3 \log_q \frac{q+2}{(S_N)^{\frac{1}{N}}}. \quad (8.4)$$

The value of S_N would give a bound on ω .

Naive combinatorial calculation:

Since any of the variables should not repeat, without loss of generality (WLOG), let A_N be the binary string of length N representing the X -coordinate of a tensor in T . Then, since A_N can only assume 2^N distinct values, we must have

$$S_N \leq 2^N. \quad (8.5)$$

But a better bound can also be obtained.

Better combinatorial calculation:

Let a tensor be obtained from a number of 0, 1, 1s, b number of 1, 0, 1s, c number of 1, 1, 0s as described before, with $a + b + c = N$. Let all the terms in T be bucketed into buckets with the corresponding unique label (a, b, c) . Then there are $\frac{(N+1)(N+2)}{2}$ buckets in total (since $a + b + c = N$). From the bucket (a, b, c) , let the coefficient of X of a chosen tensor be A_N . Then A_N can assume at most $\binom{N}{a}$ distinct values. Similarly, for coefficients Y and Z , at most $\binom{N}{b}$ and $\binom{N}{c}$ distinct coefficients can be present. Hence, from a bucket (a, b, c) , one can at most pick $\min\{\binom{N}{a}, \binom{N}{b}, \binom{N}{c}\}$ number of tensors. Since $a + b + c = N$, WLOG, $a \leq \frac{N}{3}$. Then $\min\{\binom{N}{a}, \binom{N}{b}, \binom{N}{c}\} \leq \binom{N}{a} \leq \binom{N}{\frac{N}{3}}$.

This implies that from any bucket, at most $\binom{N}{\frac{N}{3}}$ can be chosen. This further implies that

$$S_N \leq S^* = \frac{(N+1)(N+2)}{2} \binom{N}{\frac{N}{3}} \implies \omega \leq 3 \log_q \frac{q+2}{(S^*)^{\frac{1}{N}}} \quad (8.6)$$

From Stirling's approximation,

$$(S^*)^{\frac{1}{N}} = \left(\frac{\binom{(N+1)(N+2)}{2} (N!)}{\left(\frac{N}{3}\right)! \left(\frac{2N}{3}\right)!} \right)^{\frac{1}{N}} \approx \left(\frac{27}{4} \right)^{\frac{1}{3}} \text{ as } N \rightarrow \infty. \quad (8.7)$$

This implies that

$$\omega \leq 3 \log_q (q+2) \left(\frac{4}{27} \right)^{\frac{1}{3}}. \quad (8.8)$$

[2] Optimizing over q yields

$$\omega \leq \log_8 \frac{4000}{27} \implies \omega \leq 2.404, \text{ (at } q = 8\text{)}. \quad (8.9)$$

The algorithms of Strassen, Bini et al., Schonhage and Coppersmith, Winograd have been studied for their efficiency in computing matrix multiplication. It is to note that the problem of decreasing the upper bound of the exponent of matrix multiplication has been converted to the determination of rank or border rank of trilinear forms. Application of these algorithms or similar ones to special matrices (like sparse matrices etc.) or rectangular matrices to obtain better results is of interest.

Bibliography

- [1] J. Alman and V. V. Williams. A refined laser method and faster matrix multiplication. In *32nd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [2] D. Coppersmith and S. Winograd. Matrix multiplication via arithmetic progressions. *Journal of Symbolic Computation*, 9(3):251–280, 1990.
- [3] F. Romani D. Bini, M. Capovani and G. Lotti. $o(n^{2.7799})$ complexity for $n \times n$ approximate matrix multiplication. *Information Process Letters*, 8:234–235, 1979.
- [4] M. Clausen P. Burgisser and M.A. Shokrollahi. *Algebraic complexity theory*. Springer Verlag, 1997.
- [5] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.